Personal Knowledge Retrieval Assistant (PKRA) - Final Project Report

---

1. Use Case Description

1.1 Motivation and Problem Statement

In today's digital world, personal and professional knowledge is scattered across multiple file formats and domains—academic notes, project files, code repositories, presentations, and more. Manually locating precise information is time-consuming and error-prone.

Large language models (LLMs) like GPT-4 generate coherent answers but can hallucinate or provide outdated facts. Retrieval-Augmented Generation (RAG) combines the strength of external knowledge retrieval with generative models to produce factually grounded, accurate, and context-aware answers.

1.2 Project Goal

PKRA is designed to build a **custom, modular RAG system** that enables users to:

- Seamlessly query their own multi-format document collections.
- Retrieve relevant chunks of knowledge efficiently using semantic search.
- Generate precise answers grounded strictly on retrieved content.
- Benefit from metadata filtering to restrict search scope (e.g., only academic materials).

1.3 Target Users and Scenarios

- **Students:** Quickly retrieve lecture notes, assignments, and research papers.
- **Freelancers:** Access project documents, code, contracts on demand.
- **Educators:** Pull teaching materials, code examples, and presentations effortlessly.

2. Dataset and Preprocessing

2.1 Dataset Description

- Total size: Approximately **30 GB** of personal data.
- Domains: Academic, freelance, teaching.
- File types:
    - .pdf (research papers, notes)
    - .docx (reports, project documentation)
    - .ipynb (Jupyter notebooks with code and markdown)

    - .py, .cpp, .h (source code)
    - .pptx (presentations)
    - .txt (plain text/code snippets)
    - .csv, .xlsx (tabular data)

2.2 File Indexing (file_indexer.py)

- **Purpose:** Generate metadata describing every relevant file.
- **Process:**
    - Recursively scan base directory.
    - Filter files by target extensions.
    - Extract metadata:
        - Filename, relative path (normalized for cross-platform).
        - File type: “code” for source files, “document” otherwise.
        - Extension.
        - Domain metadata (source and category) mapped from top-level folders.
    - Save metadata as indexed_metadata.json.
- **Parameters:**
    - BASE_DIR: Root path to dataset.
    - TEXT_EXTENSIONS: Defines file types indexed.
    - SOURCE_CATEGORY_MAP: Maps root folders to semantic categories.
- **Significance:**
    - Enables **domain-aware retrieval filtering**.
    - Provides foundational metadata for downstream processing.

2.3 Content Extraction (content_extractor_v2.py)

- **Objective:** Extract readable text content from all files.

- **Supported formats and libraries:**

Format	Extraction Tool	Method
.pdf	PyMuPDF (fitz)	Page-wise text extraction
.docx	python-docx	Paragraph concatenation
.ipynb	nbformat	Extract markdown + code cells
.pptx	python-pptx	Extract text from shapes and slides
.csv, .xlsx	pandas	Read and stringify tables
.txt, .py, .cpp, .h	Standard UTF-8 read	Raw text read
.html	BeautifulSoup	Visible text extraction

- **Error Handling:**
  Each extractor returns error messages embedded in text to avoid crashing the pipeline.
- **Output:**
  Saves enriched file metadata with a "text" field as enriched_with_text.json.
- **Why important:**
  Normalizes heterogeneous files to a uniform textual representation required for embedding.

2.4 Semantic Chunking (semantic_chunker.py)

- **Rationale:** Large documents exceed embedding model token limits and reduce semantic focus.
- **Approach:**
  - Use NLTK to tokenize text into sentences.
  - Aggregate sentences into chunks bounded by min_chunk_size=200 and max_chunk_size=800 characters.
  - Preserve sentence boundaries and avoid arbitrary splitting.
- **Output:** List of semantically coherent chunks per document.
- **Parameters:**
  - min_chunk_size, max_chunk_size tunable for different datasets or models.

3. System Design and Architecture

3.1 Modular Pipeline

PKRA's workflow consists of the following sequential modules:

Step	Description	Script/Component
Document Ingestion	File indexing + text extraction	file_indexer.py, content_extractor_v2.py
Semantic Chunking	Split raw text into meaningful chunks	semantic_chunker.py
Embedding & Storage	Embed chunks, store vectors with metadata	semantic_chroma_indexer.py
Retrieval	Semantic search with metadata filtering	search_chroma.py
Reranking	Cross-encoder re-scoring for relevance	rerank_results.py
Answer Generation	GPT-4o-mini generation with context prompt	rag_generator.py
Orchestration	CLI integration for pipeline execution	cli_rag_assistant.py

3.2 Embeddings and Vector Storage

- **Model:** all-mpnet-base-v2 from SentenceTransformers for high semantic accuracy.
- **Hardware:** CUDA-enabled GPU for batch embedding acceleration.
- **Vector Store:** ChromaDB with persistent storage and metadata filtering.
- **Metadata Stored:** Filename, path, source, category, file type, extension, chunk index.
- **UUIDs:** Unique chunk IDs generated for indexing and traceability.

3.3 Retrieval

- Embeds user query using same model for consistency.
- Retrieves top-30 chunks from ChromaDB by cosine similarity.

- Interactive filters for filename, source, category, and extension narrow results.
- Saves retrieval output for reranking.

3.4 Reranking

- Cross-encoder model cross-encoder/ms-marco-MiniLM-L-12-v2 jointly scores query and chunks.
- Filters out chunks scoring < 2.0.
- Selects top 10–15 for answer generation.
- Balances precision and recall, improving answer quality.

3.5 Answer Generation

- Constructs prompt combining reranked chunks with labels [rank] filename (path).
- Uses OpenAI GPT-4o-mini with temperature=0.5, max_tokens=512.
- Encourages model to answer only from context or reply "I don't know."
- Efficient in API usage — approx $0.20 per million prompts, cost-effective for personalized data scales.

3.6 CLI Orchestration

- Interactive command line tool guiding user queries through filtering, retrieval, reranking, and generation.
- Robust logging captures errors and info.
- Saves intermediate results for transparency and debugging.

4. Retrieval and Generation Components (Technical Details)

4.1 Semantic Search (search_chroma.py)

- Embeds queries with HuggingFaceEmbeddings wrapper.
- Query method allows metadata filtering using ChromaDB's .query() function with where clause.
- Results include document text and metadata for display.
- Filters significantly improve search relevance for large corpora.

4.2 Reranker (rerank_results.py)

- Loads retrieved chunks and query JSON.
- Cross-encoder initialized with GPU fallback to CPU.
- Batch predicts relevance scores.
- Scores thresholded at 2.0; top-k chunks sorted and saved.

4.3 Generator (rag_generator.py)

- Loads reranked JSON.
- Builds prompt with context concatenation, labels for traceability.
- Calls OpenAI API with chat completions endpoint.
- Parameters tuned for balance between creativity and factuality.

5. Evaluation and Results

5.1 Qualitative Examples

- Queries related to academic notes return precise answers citing source files.
- Metadata filters restrict search domain, e.g., only “teaching” materials.
- Cross-encoder reranking filters noisy results, improving final answer coherence.

5.2 Performance Metrics

- Average retrieval time under 1 second for 30 top chunks.
- Reranking latency around 2-3 seconds on GPU.
- Generation response typically under 2 seconds.
- API costs maintained low with efficient chunk limits and small GPT model.

6. Challenges and Limitations

- Parsing complex notebooks and code files requires manual fixes.
- Reranking latency increases with candidate size; scaling needs optimization.
- Overlapping chunks can cause repetition in generated answers.
- Dataset updates require manual re-extraction and reindexing.
- External API dependency limits offline use and incurs cost.

7. Future Work

- Build user-friendly web UI with Streamlit integrating filters and explanations.
- Incorporate hybrid retrieval combining sparse (keyword) and dense (semantic) search.
- Automate document classification and metadata enrichment with ML classifiers.
- Fine-tune reranker and generation models on your personalized 30 GB data.
- Add ingestion from cloud storage and email services.
- Implement confidence scoring and explicit citation in answers for traceability.

8. Conclusion

PKRA offers a comprehensive, efficient RAG system for personal knowledge management by integrating multi-format ingestion, semantic chunking, scalable vector search, precision reranking, and GPT-powered answer generation. The system is cost-effective, scalable, and provides reliable, context-aware answers grounded in a user's own documents.

Appendix

Code Summary

- **file_indexer.py**: Metadata indexing of dataset files.
- **content_extractor_v2.py**: Text extraction from diverse file types.
- **semantic_chunker.py**: Sentence-based semantic chunking.
- **semantic_chroma_indexer.py**: Chunk embedding and ChromaDB vector storage.
- **search_chroma.py**: Semantic search with metadata filters.
- **rerank_results.py**: Cross-encoder reranking of retrieved chunks.
- **rag_generator.py**: GPT-4o-mini based answer generation.
- **cli_rag_assistant.py**: Interactive pipeline orchestration and CLI.

Sample Inputs/Outputs

project code > No index.py > ...

```python
SOURCE_CATEGORY_MAP = {
    "ucp": ("academic", "semester_study"),
    "coding files": ("academic", "code_assignments"),
    "py online class": ("teaching", "python_teaching"),
    "projects": ("freelance", "project"),
    "course + books material": ("personal", "learning_material"),
}

# Target file types
TEXT_EXTENSIONS = {'.txt', '.py', '.ipynb', '.pdf', '.docx', '.cpp', '.h'}

indexed_files = []

print(f" Scanning files under: {BASE_DIR}\n")

for root, _, files in os.walk(BASE_DIR):
    for file in files:
        full_path = Path(root) / file
        ext = full_path.suffix.lower()

        if ext in TEXT_EXTENSIONS:
            rel_path = full_path.relative_to(BASE_DIR)
            parts = rel_path.parts
            top_folder = parts[0].lower().strip() if parts else "unknown"

            # Match folder to source/category
            source, category = SOURCE_CATEGORY_MAP.get(top_folder, ("unknown", "uncategorized"))

            indexed_files.append({
                "filename": file,
                "path": str(rel_path).replace("\\", "/"),
                "file_type": "code" if ext in {'.py', '.ipynb', '.cpp', '.h'} else "document",
                "extension": ext,
                "source": source,
                "category": category
            })
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
Indexed: ucp\6th semester\toa\A3-TOA.docx
Indexed: ucp\6th semester\toa\Course-Description Theory of Automata.docx
Indexed: ucp\6th semester\toa\Hopcroft-Motwani-Ullman-2001.pdf
Indexed: ucp\6th semester\toa\Introduction-To-The-Theory-Of-Computation-Michael-Sipser.pdf
Indexed: ucp\6th semester\toa\NFA to DFA Conversion.docx
Indexed: ucp\6th semester\toa\nury_hsbt_1th_lthny.pdf
Indexed: ucp\6th semester\toa\Short-TOA.docx
Indexed: ucp\6th semester\toa\Theory of Automata (TOA) Lecture 13   .pdf
Indexed: ucp\6th semester\toa\Theory of Automata (TOA) Lecture 14   .pdf
Indexed: ucp\6th semester\toa\Theory of Automata (TOA) Lecture 6   .pdf
Indexed: ucp\6th semester\toa\TOA-1.docx
Indexed: ucp\course outlines\Big Data Analytics Course Outline Oct 2022 (1).pdf
Indexed: ucp\course outlines\Course Log Artificial Intelligence Fall_2022.docx
Indexed: ucp\course outlines\intro to ml.pdf
Indexed: ucp\course outlines\MAD Course log File.pdf
Indexed: ucp\course outlines\OR Course Outline.pdf
Indexed: ucp\fyp\HAMZA PROPOSAL.docx
Indexed: ucp\fyp\HAMZA PROPOSAL.pdf
Indexed: ucp\fyp\haris , smart recruit project .docx
Indexed: ucp\fyp\INTELLIHIRE.docx
Indexed: ucp\fyp\INTELLIHIRE.pdf
Indexed: ucp\fyp\proposal.docx
Indexed: ucp\fyp\~$ris , smart recruit project .docx

Total indexed: 10630 files
Saved metadata to: D:\project ir\dataset for ir\upload_metadata.json
```

project code > content_extractor_v2.py > ...

```python
import json
import fitz  # PyMuPDF
import nbformat
import pandas as pd
from pathlib import Path
from docx import Document
from pptx import Presentation
from bs4 import BeautifulSoup

# === Paths ===
BASE_DIR = Path(r"D:\project ir\dataset for ir")
METADATA_PATH = Path(r"D:\project ir\uploaded_metadata.json")
OUTPUT_PATH = Path(r"D:\project ir\enriched_with_text.json")

def read_text_file(path):
    try:
        return Path(path).read_text(encoding="utf-8", errors="ignore")
    except Exception as e:
        return f"[ERROR reading text: {e}]"

def read_pdf(path):
    try:
        text = ""
        with fitz.open(path) as doc:
            for page in doc:
                text += page.get_text()
        return text
    except Exception as e:
        return f"[ERROR reading PDF: {e}]"

def read_docx(path):
    try:
        doc = Document(path)
        return "\n".join([p.text for p in doc.paragraphs])
    except Exception as e:
        return f"[ERROR reading DOCX: {e}]"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\project ir> & C:/Users/ARIFr/AppData/Local/Programs/Python/Python312/python.exe "d:/project ir/project code/content_extractor_v2.py"
 Processed: coding files/ammar c++/arrays.cpp
 Processed: coding files/ammar c++/conditionals.cpp
 Processed: coding files/ammar c++/dowhile.cpp
 Processed: coding files/ammar c++/nestedloop.cpp
 Processed: coding files/ammar c++/operators.cpp
 Processed: coding files/ammar c++/practice.cpp
 Processed: coding files/ammar c++/switch.cpp
 Processed: coding files/ammar c++/task61ab7.cpp
 Processed: coding files/ammar c++/ternary operator.cpp
 Processed: coding files/ammar c++/while loop.cpp
 Processed: coding files/graphs/bfs.cpp
 Processed: coding files/graphs/Edge.cpp
 Processed: coding files/graphs/Edge.h
 Processed: coding files/graphs/Graph.cpp
 Processed: coding files/graphs/Graph.h
 Processed: coding files/graphs/graph.txt
 Processed: coding files/graphs/graphstesting.cpp
 Processed: coding files/graphs/test.cpp
 Processed: coding files/graphs 2/Project1/bfs.cpp
 Processed: coding files/graphs 2/Project1/Edge.cpp
 Processed: coding files/graphs 2/Project1/Edge.h
 Processed: coding files/graphs 2/Project1/Graph.cpp
 Processed: coding files/graphs 2/Project1/Graph.h
 Processed: coding files/graphs 2/Project1/graph.txt
 Processed: coding files/graphs 2/Project1/test3.cpp
 Processed: coding files/hangman game/pfprj.cpp
```

project code > semantic_chroma_indexer.py > ...

```python
import json
import uuid
import chromadb
from pathlib import Path
from sentence_transformers import SentenceTransformer
from semantic_chunker import semantic_chunk

# === Paths ===
INPUT_PATH = Path(r"D:\project ir\enriched_with_text.json")
CHROMA_DIR = Path(r"D:\project ir\chroma_db_semantic")
CHROMA_DIR.mkdir(parents=True, exist_ok=True)

# === Load data ===
with open(INPUT_PATH, "r", encoding="utf-8") as f:
    files = json.load(f)

# === Setup ChromaDB and embedder ===
chroma_client = chromadb.PersistentClient(path=str(CHROMA_DIR))
collection = chroma_client.get_or_create_collection(name="rag-files-semantic")
embedder = SentenceTransformer("all-mpnet-base-v2", device="cuda")


chunk_total = 0

for file in files:
    text = file.get("text", "")
    if not text.strip():
        continue

    try:
        chunks = semantic_chunk(text)
    except Exception as e:
        print(f" Skipped {file['path']} due to chunking error: {e}")
        continue
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\project ir> & C:/Users/ARIFr/AppData/Local/Programs/Python/Python312/python.exe "d:/project ir/project code/semantic_chroma_indexer.py"
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ARIFr\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
Indexed: coding files/ammar c++/conditionals.cpp (1 semantic chunks)
Indexed: coding files/ammar c++/nestedloop.cpp (1 semantic chunks)
Indexed: coding files/ammar c++/switch.cpp (1 semantic chunks)
Indexed: coding files/ammar c++/task6lab7.cpp (1 semantic chunks)
Indexed: coding files/ammar c++/ternary operator.cpp (1 semantic chunks)
Indexed: coding files/graphs/bfs.cpp (1 semantic chunks)
Indexed: coding files/graphs/Edge.cpp (1 semantic chunks)
Indexed: coding files/graphs/Edge.h (1 semantic chunks)
Indexed: coding files/graphs/Graph.cpp (1 semantic chunks)
Indexed: coding files/graphs/Graph.h (1 semantic chunks)
Indexed: coding files/graphs/test.cpp (1 semantic chunks)
Indexed: coding files/graphs 2/Project1/bfs.cpp (1 semantic chunks)
Indexed: coding files/graphs 2/Project1/Edge.cpp (1 semantic chunks)
Indexed: coding files/graphs 2/Project1/Edge.h (1 semantic chunks)
Indexed: coding files/graphs 2/Project1/Graph.cpp (1 semantic chunks)
Indexed: coding files/graphs 2/Project1/Graph.h (1 semantic chunks)
Indexed: coding files/graphs 2/Project1/test3.cpp (1 semantic chunks)
Indexed: coding files/hangman game/pfpr1.cpp (4 semantic chunks)
Indexed: coding files/hangman game/Source.cpp (1 semantic chunks)
Indexed: coding files/hangman game/Source1.cpp (1 semantic chunks)
Indexed: coding files/hangman game/Source2.cpp (3 semantic chunks)
Indexed: coding files/hangman game/Source3.cpp (2 semantic chunks)
```

project code > semantic_chunker.py > ...

```python
import os
import json
import nltk
from sentence_transformers import SentenceTransformer
from tqdm import tqdm

nltk.download('punkt')

# Load model with GPUs
model = SentenceTransformer("all-mpnet-base-v2", device="cuda")

def split_into_sentences(text):
    return nltk.sent_tokenize(text)

def semantic_chunk(text, min_chunk_size=200, max_chunk_size=800):
    sentences = split_into_sentences(text)
    chunks = []
    current_chunk = ""

    for sentence in sentences:
        if len(current_chunk) + len(sentence) <= max_chunk_size:
            current_chunk += " " + sentence
        else:
            if len(current_chunk.strip()) >= min_chunk_size:
                chunks.append(current_chunk.strip())
            current_chunk = sentence

    if len(current_chunk.strip()) >= min_chunk_size:
        chunks.append(current_chunk.strip())

    return chunks

def load_json_file(path):
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\project ir> & C:/Users/ARIFr/AppData/Local/Programs/Python/Python312/python.exe "d:/project ir/project code/semantic_chunker.py"
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ARIFr\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
Semantic Chunking: 100%|[redacted]| 10630/10630 [00:15<00:00, 671.26it/s]
Done. Saved 167922 semantic chunks to semantic_chunks.json
PS D:\project ir>
```

{} semantic.json > [] chunks > {} 0 > {} metadata > ⊟ file_type

```
{
  "query": "python",
  "chunks": [
    {
      "chunk": "TYPE -4\n\nPython Basics: Getting Started\n\nMain Python Data Types\n\nHow to Create a String in Python\n\nMath Operators\n\nHow to Store Strings in Variables\n\nBuilt-in Functions in Python\n\nHow to Define a Function\n\nlist\n\nlist Comprehensions\n\ntuples\n\
      "metadata": {
        "extension": ".pdf",
        "source": "personal",
        "filename": "Python_Cheatsheet_1706426535.pdf",
        "path": "course + books material/imp books from linkedin/Python_Cheatsheet_1706426535.pdf",
        "category": "learning_material",
        "chunk_index": 22,
        "file_type": "document"
      },
      "rank": 1,
      "score": 0.0
    },
    {
      "chunk": "Processes and things have become automated, programmable, \nsmaller, faster, highly interconnected, and easily visualized and interpreted. If you want to learn more about the history of computing, consult the sources listed at the \nend of
      "metadata": {
        "source": "personal",
        "extension": ".pdf",
        "filename": "Python fundamentals.pdf",
        "path": "course + books material/python books/Python fundamentals.pdf",
        "file_type": "document",
        "category": "learning_material",
        "chunk_index": 158
      },
      "rank": 2,
      "score": 0.0
    },
    {
      "chunk": "AUTOMATE THE BORING STUFF WITH\nPYTHON\n2ND EDITION\nPractical Programming for Total Beginners\nby Al Sweigart\nSan Francisco\nAUTOMATE THE BORING STUFF WITH PYTHON, 2ND EDITION. Copyright ©\n2020 by Al Sweigart. All rights reserved. No par
      "metadata": {
        "category": "learning_material",
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Enter your query: python

Optional: Apply filters
Do you want to apply filters? (y/n): n

Top 10 Results:
--------------------------------------------------

Result 1:
File: Python_Cheatsheet_1706426535.pdf
Path: course + books material/imp books from linkedin/Python_Cheatsheet_1706426535.pdf
Source: personal, Category: learning_material
Snippet: TYPE -4
Python Basics: Getting Started
Main Python Data Types
How to Create a String in Python
Math Operators
How to Store Strings in Variables
Built-in Functions in Python
How to Define a Function
list
list Comprehensions
tuples
Dictionaries
If Statements (Conditional Statements) in Python
```

rag_reranked.json > [ ] chunks > {} 1 > {} metadata > path


```
{
  "query": "python",
  "chunks": [
    {
      "chunk": "Introduction to Python and installation: \nPython is a widely used general-purpose, high level programming language. It was initially \ndesigned by Guido van Rossum in 1991 and developed by Python Software Foundation. It \nwas mainly develo
      "metadata": {
        "extension": ".pdf",
        "category": "learning_material",
        "chunk_index": 7,
        "path": "course + books material/python books/PYTHON PROGRAMMING NOTES.pdf",
        "file_type": "document",
        "source": "personal",
        "filename": "PYTHON PROGRAMMING NOTES.pdf"
      },
      "rank": 8,
      "score": 8.831155776977539
    },
    {
      "chunk": "Python also includes all of the advanced features of a modern programming language, \nsuch as support for data structures and object-oriented software development, for use \nwhen they become necessary. ••\nPython is highly interactive. Expr
      "metadata": {
        "file_type": "document",
        "category": "learning_material",
        "filename": "Python Fundamentals.pdf",
        "source": "personal",
        "path": "course + books material/python books/Python Fundamentals.pdf",
        "extension": ".pdf",
        "chunk_index": 36
      },
      "rank": 16,
      "score": 7.979561805725098
    },
    {
      "chunk": "1.1[ HISTORY OF PYTHON\nPython was created by Guido Van Rossum in the early nineteenth century at the national research institute for mathematics and computer \nscience in the Netherlands. Python is obtained from several additional \nprog
      "metadata": {
        "category": "learning_material",
```


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\project ir> & C:/Users/ARIFr/AppData/Local/Programs/Python/Python312/python.exe "d:/project ir/project code/rerank_results.py"

 Reranking 30 chunks for query: "logical operator"

 Saved reranked top 15 chunks to rag_reranked.json
PS D:\project ir> & C:/Users/ARIFr/AppData/Local/Programs/Python/Python312/python.exe "d:/project ir/project code/rerank_results.py"

 Reranking 30 chunks for query: "python"

 Saved reranked top 15 chunks to rag_reranked.json
PS D:\project ir>
```

project code > rag_generator.py > ...

```
MODEL = "gpt-3.5-turbo"  # You can also use "gpt-4o-mini" or "gpt-3.5-turbo"

def load_reranked_chunks(file_path="rag_reranked.json"):
    with open(file_path, "r", encoding="utf-8") as f:
        data = json.load(f)
    return data["query"], data["chunks"]

def build_prompt(query, chunks):
    context = ""
    for entry in chunks:
        label = f"[{entry['rank']}] {entry['metadata']['filename']} ({entry['metadata']['path']})"
        doc = entry["chunk"].strip()
        context += f"{label}:\n{doc}\n\n"

    return f"""You are a helpful assistant. Use the following context to answer the question.
If the answer is not in the context, reply with "I don't know."

Context:
{context}
Question: {query}
Answer:"""

def generate_with_openai(prompt):
    response = client.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": "You are a helpful assistant that answers questions based only on the context provided."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.5,
        max_tokens=512
    )
    return response.choices[0].message.content.strip()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\project ir> & C:/Users/ARIFr/AppData/Local/Programs/Python/Python312/python.exe "d:/project ir/project code/rag_generator.py"
 RAG Generation using OpenAI API (v1.x)
 Generating answer from OpenAI...

 Answer:
Python is a widely used general-purpose, high-level programming language that emphasizes code readability and allows programmers to express concepts in fewer lines of code. It was initially designed by Guido van Rossum in 1991 and developed by Python Software Four
has two major versions, Python 2 and Python 3. It is highly interactive, general-purpose, free, and used in various fields such as web programming, application testing, and data science.
PS D:\project ir>
```

file_indexer.py | content_extractor_v2.py | semantic_chunker.py | search_chroma.py | rerank_results.py | cli_rag_assistant.py | llm_n_query_expansion.py | semantic_chroma_indexer.py | rag_generator.py | rag_reranked.json

cli_rag_assistant.py > ...

```
import os
import json
import subprocess
import logging
from sentence_transformers import CrossEncoder
from langchain_community.embeddings import HuggingFaceEmbeddings
import chromadb
import torch

# === Logging Setup ===
logging.basicConfig(
    filename="rag_pipeline.log",
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s"
)

# === Configuration ===
CHROMA_DIR = r"D:\project ir\chroma_db_semantic"
COLLECTION_NAME = "rag-files-semantic"
EMBED_MODEL = "all-mpnet-base-v2"
CROSS_ENCODER_MODEL = "cross-encoder/ms-marco-MiniLM-L-12-v2"
TOP_K_RETRIEVE = 30
TOP_K_FINAL = 10
SCORE_THRESHOLD = 2.0
GENERATOR_PATH = r"D:\project ir\project code\rag_generator.py"

# === Load Models ===
logging.info("Loading Chroma collection and models.")
chroma_client = chromadb.PersistentClient(path=CHROMA_DIR)
collection = chroma_client.get_collection(name=COLLECTION_NAME)

embedder = HuggingFaceEmbeddings(
    model_name=EMBED_MODEL,
    model_kwargs={"device": "cuda"}
)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\project ir> & C:/Users/ARIFr/AppData/Local/Programs/Python/Python312/python.exe "d:/project ir/cli_rag_assistant.py"
d:\project ir\cli_rag_assistant.py:32: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain 0.2.2 and will be removed in 1.0. An updated version of the class exists in the :class:`~langchain-huggingface package and should be u
  use it run `pip install -U :class:`~langchain-huggingface` and import as `from :class:`~langchain_huggingface import HuggingFaceEmbeddings``.
  embedder = HuggingFaceEmbeddings(
🔎 One-Stop RAG Pipeline Interface

🔍 Enter your question (or 'exit'): what is input function
Do you want to use metadata filters? (y/n): n
Batches: 100%|##########################################################################################| 1/1 [00:00:00]
 Saved reranked chunks to rag_reranked.json
 Generating answer...

 Answer:
RAG Generation using OpenAI API (v1.x)
 Generating answer from OpenAI...

 Answer:
The input function in Python is used to prompt the user to enter some text, which is then assigned to a variable for further processing. The input function reads the data entered by the programmer and returns it as a string.

🔍 Enter your question (or 'exit'):
```